Edge AI-driven Robotic Applications in the Computing Continuum

Nikos Filinis*, Dimitrios Spatharakis*, Ioannis Dimolitsas*, Gaurav Kapoor†, Lei Fu†, Giovanni Toffetti†, Leonardo Militano†, Thomas M. Bohnert†, Anastasios Zafeiropoulos*, Member, IEEE, Symeon Papavassiliou*, Senior Member, IEEE

*School of Electrical and Computer Engineering, National Technical University of Athens, Greece
{nfilinis,dspatharakis,jdimol}@netmode.ntua.gr, tzafeir@cn.ntua.gr, papavass@mail.ntua.gr

†ZHAW Zurich University of Applied Sciences 8400 Winterthur, Switzerland
{kapu, fule, toff, milt, bohe}@zhaw.ch.

***Abstract*—Internet of Robotics Things (IoRT) shifts from traditional robotic operations to a paradigm shift that integrates resource-constrained robots as part of the Computing Continuum, providing additional resources and more advanced application capabilities. Edge Artificial Intelligence (AI) plays a pivotal role in this transition, facilitating real-time decision-making and timely adaptation in response to changes in the robot's environment. In this paper, we propose a framework for the implementation of Edge AI in the IoRT, compatible with various IoRT scenarios. Robot resources are integrated into a unified orchestration platform, facilitating seamless and on-demand deployment of application nodes at the edge of the network in an event-triggered manner. An AI-assisted task offloading mechanism is proposed to cater to the needs of real-time adaptation of an inference pipeline based on the robot's state. Experimental results indicate that the framework outperforms baseline techniques in terms of accuracy of the tracking algorithm and a significant reduction in energy consumption of the edge resources due to the n-demand deployment.**

***Index Terms*—Internet of Robotic Things, Computing Continuum, Task Offloading, Edge AI, Edge Inference**

I. Introduction

The immense growth of the Internet of Things (IoT) and the penetration of Artificial Intelligence (AI) across various industries are driving the evolution of novel robotic applications. The IoRT emerges as a paradigm shift, enabling smart robots to integrate various sources of information and interact seamlessly with their environment [1]. The IoRT applications face significant challenges compared to traditional IoT, as the inherent mobility of robots, the need to process large volumes of data, and the highly dynamic environments in which robots operate require real-time decision-making and advanced AI algorithms to perform complex missions [2].

IoRT applications often perform computationally intensive tasks, such as AI inference, and require dedicated hardware accelerators to ensure high performance. Edge Computing is becoming a standard practice to provide additional resources closer to the resource and battery-constrained robots. This concept, also known as Edge/Fog robotics, necessitates the seamless integration of robots with the edge and cloud parts of the network by forming a unified continuum of available computing infrastructure, known as the Computing Continuum. Recent studies on device-edge-cloud resource management have highlighted hierarchical task offloading as a key mechanism for IoT systems, demonstrating that AI-based methods are promising to overcome the limitations of traditional optimization under dynamic latency and resource constraints [3]. Such offloading strategies allow robots to autonomously offload computation-intensive tasks to nearby edge servers equipped with GPUs, reducing communication overhead while ensuring real-time performance and uninterrupted operation.

Edge AI provides effective solutions to address both the computational and communication challenges encountered in IoRT applications, with Federated Learning (FL) playing a key role to enable distributed intelligence across heterogeneous devices, including robots and other resource- and power-constrained systems [4]. However, deploying AI on-device equipment requires lightweight and energy-efficient algorithms to optimize power consumption and extend the autonomy of robots. However, such AI algorithms may provide limited accuracy and disrupt the smooth execution of critical missions, such as search and rescue operations [5]. As a result, the development of TinyML models optimized for robotic platforms, while offloading resource-intensive, however accurate, algorithms to the edge infrastructure, is crucial for balancing accuracy and efficiency.

To address these challenges, it is of paramount importance to employ intelligent mechanisms that orchestrate Edge AI applications by co-designing the dynamic network conditions, the available computing resources, along with the state of the robotic mission and the robot [6]. To this end, AI and Machine Learning (ML) have proven to be resilient in identifying unknown states and capturing dynamic behaviors toward real-time and adaptive decision making for energy-efficient computation.

Moreover, in the context of IoRT, robots need to exchange a vast amount of information with many Robotic Operating System (ROS) nodes and application nodes in a device-independent way. As a result, the concept of Virtual Object (VO) is utilized to continuously monitor the state of the robot, interact and control with the robotic hardware of the robot, while at the same time dealing with interoperability issues with the application layer [7]. Therefore, the VO is capable of acting

as an intelligent unit deployed on the robot that controls the robot in real-time, and augments the supported functionalities by seamlessly interacting with the application logic.

The realization of Edge/Cloud robotics and modern IoRT applications inherently relies on optimized decisions for the task offloading problem. When properly designed, task offloading can significantly improve the overall performance of robotic operations. Specifically, for optimized Edge AI inference, a smart offloading strategy should select the appropriate AI model (local or remote) based on the state of the robot and identify trade-offs between minimizing the overall inference latency and maximizing accuracy while also reducing the energy consumption of the robot. However, these goals are inherently conflicting.

This paper introduces a novel architecture for the implementation of Edge AI for IoRT applications. We select an inference pipeline as the Edge AI application, which is applicable in many IoRT scenarios. The proposed architecture equips a platform tailored for the unified orchestration of resources in the context of the Computing Continuum. Moreover, we employ a Reinforcement Learning (RL)-based solution aiming to address the multi-objective optimization problem that captures the state of the robot and dictates the task offloading process to optimize the overall performance and minimize the power utilization on the robot. The main contributions of the paper are as follows:

- We introduce an architecture for the implementation of Edge AI, paving the way for seamless interaction of robots with the application layer in the context of edge robotics, taking advantage of an open source software stack that we have developed for the virtualization of devices in the Internet of Things (IoT) [8]. We utilize an orchestration platform that continuously monitors the robot's state and coordinates the deployment of additional resources based on real-time events generated by the robot. A VO that acts as a digital replica of the robot and is deployed on the robot to control the operations and monitor the robot's state.
- The resource-constrained robot equips a low-power AI algorithm and seeks additional resources from the edge infrastructure only if required. To optimize the offloading process, we formulate an RL-based agent deployed on the robot that controls the number and compression rate of offloaded tasks. The proposed agent also considers the state of the robot and tries to minimize the energy consumption of the robot while maximizing the accuracy of the inference and tracking processes.
- We provide a realistic implementation on a small-scale testbed. Evaluation results highlight that the proposed framework achieves at least a 17% increase in accuracy of the inference and tracking processes compared to baseline solutions. Moreover, the orchestration platform achieves 8% reduction in terms of energy utilization compared to static edge offloading solutions by utilizing the event-triggered deployment mechanism.

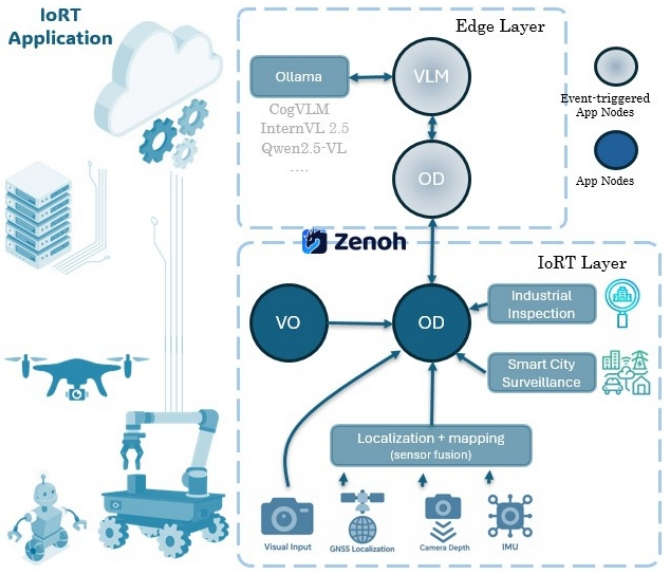


Fig. 1: IoRT Inference Application Graph.

II. IoRT Use-Case, Challenges and Motivations

The proposed solution applies to various real-world Internet of Robotic Things (IoRT) scenarios, in which one or more resource-constrained robotic agents, equipped with camera modules, capture their surroundings for an inference application (e.g., person tracking, object detection (OD)). Due to limited onboard resources, these agents execute lightweight AI models for real-time detection, as less computationally demanding algorithms help extend battery life. However, such simplified models typically suffer from reduced accuracy in inference, while the number of tasks that can be executed concurrently is limited. To address this trade-off, the robot may offload inference tasks to an edge server equipped with GPU resources to seek better accuracy, or in case it is required to process more information. Furthermore, the edge computing infrastructure is able to deploy more advanced and accurate AI models to process the captured data and extract higher-level semantic insights (e.g., tracking, posture detection, or health assessment) and enable interactions with the end-user. A significant challenge is that the robot requires additional edge resources only occasionally, making dynamic orchestration necessary. This means that the aforementioned applications should be available on demand, and their deployment should be event-triggered only when required by the robot. Therefore, it is important to monitor the robot's and application state to generate such events towards the infrastructure. Subsequently, a digital replica of the robot, namely, the VO, is able to monitor the robot's state and can generate an event towards the edge infrastructure to trigger the deployment of additional resources. A high-level overview of an example inference application graph is illustrated in Fig. 1, where the application nodes on the edge layer are conditionally deployed based on the respective triggering conditions.

Several practical scenarios justify a hierarchical, two-level inference architecture:

- **Search and Rescue:** Drones scan disaster zones for

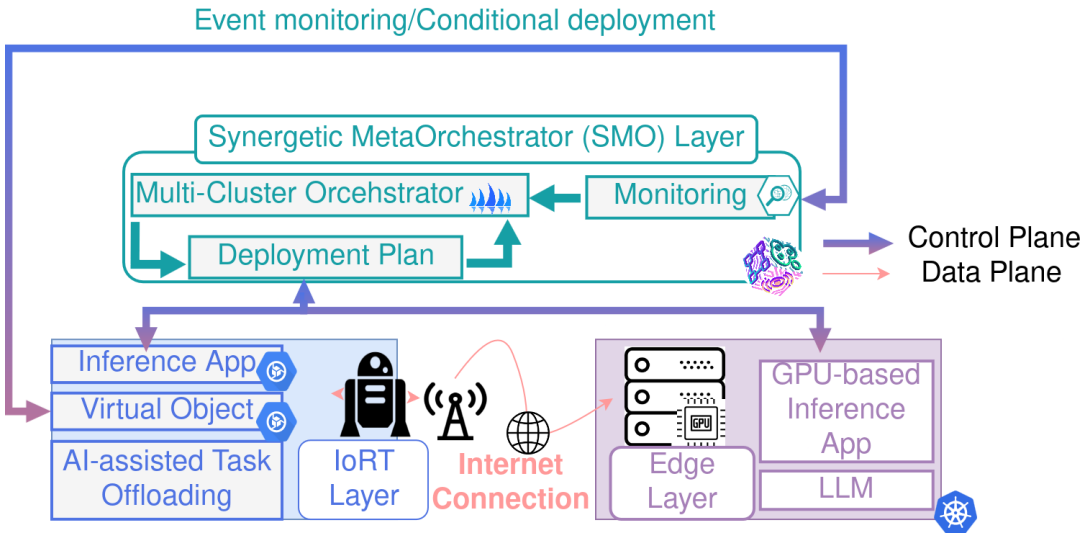


Fig. 2: Edge AI Architecture for IoRT.

survivors, while cloud models refine detections (e.g., heat signatures, distress signals).

- **Wildlife Monitoring and Anti-Poaching:** Drones detect humans or animals, while cloud inference distinguishes rangers from poachers and identifies endangered species.
- **Agriculture and Livestock:** Drones monitor crops and livestock, while cloud models assess plant diseases, estimate yields, or track missing animals.
- **Smart Cities:** Drones observe traffic and public areas, while cloud analysis recognizes vehicle types, license plates, and abnormal behavior.
- **Event Security:** Drones oversee large gatherings; cloud models detect threats, crowd density issues, or abandoned objects.
- **Industrial Inspection:** Drones capture infrastructure images, while cloud inference identifies cracks, corrosion, or other defects.

These diverse scenarios highlight the potential of Edge AI to enhance robotic capabilities, but they also expose several limitations [9] in current IoRT systems. To make such deployments practical and reliable, the following key challenges must be addressed:

- **Hardware Limitations:** Robotic agents have inherent hardware limitations, particularly in computation, storage, and energy. These limitations directly affect the achievable performance regarding inference accuracy and latency, constrained by the AI models that can be executed locally.
- **Offloading Challenges:** Designing efficient offloading strategies for Edge AI is challenging, as robots must dynamically decide between local and remote execution while trying to identify tradeoffs between latency, accuracy, and energy consumption under dynamic operating conditions.
- **Resource Heterogeneity:** Achieving unified management of computational and communication resources across robots and edge infrastructure is difficult, requiring real-time monitoring and coordination of heterogeneous devices.
- **Dynamic Resource Adaptation:** Enabling automated, event-driven deployment remains a major challenge, since real-time offloading decisions are triggered based on the robot's internal parameters (i.e., battery level, computational load). An orchestration platform should provide dynamic deployment of resources to optimize the utilization of the edge infrastructure.

III. Edge AI Architectural Framework for IoRT

Specifically, in this paper, a three-layered architecture is considered, namely (a) the IoRT Layer, (b) the Edge Layer, and (c) the Synergetic Meta-Orchestrator (SMO) Layer. The high-level architecture of the proposed framework is presented in Fig. 2.

In the **IoRT Layer**, the robot can deploy containerized applications to enable the unified management of resources by a centralized orchestrator. The robot is capable of hosting a lightweight version of state-of-practice container resource management tools (e.g., Minikube, K3s). In this paper, we consider an AI-assisted inference application graph for real-time person detection and tracking. The robot is equipped with a camera that captures the robot's surroundings and can execute a lightweight version of the inference application locally. Moreover, the corresponding VO of the robot is deployed on the device premises, able to communicate with the edge layer through a wireless access point to offload the data frames and also expose events and metrics of the robot's state. The VO offers interoperability and portability to heterogeneous robots as it abstracts the properties, actions, and events of the robot using the WoT Thing Description [1]. Moreover, the VO can enhance the robot's functionalities and continuously monitor the robot's state to decide whether to run the local detection application or offload the computation to the edge layer. Once a person is detected, computation is

[1]https://www.w3.org/TR/wot-thing-description11/

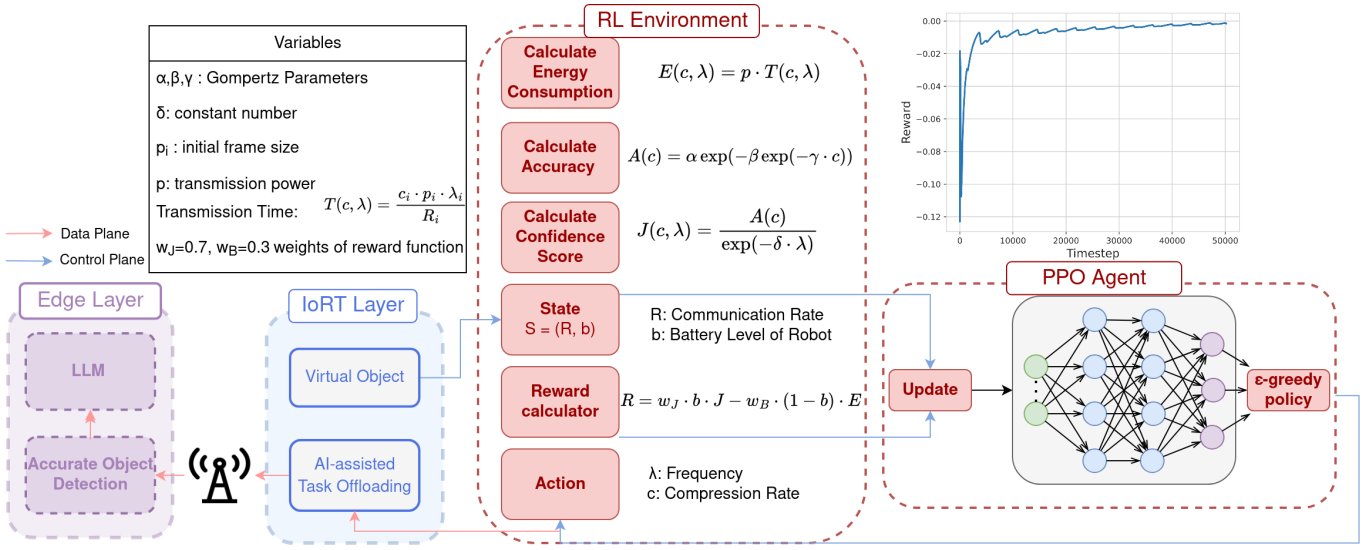


Fig. 3: Edge AI Framework for IoRT showcasing the interaction between the Edge and IoRT layers, highlighting the role of the RL Agent in the offloading process, and presenting the reward convergence during the RL Agent training.

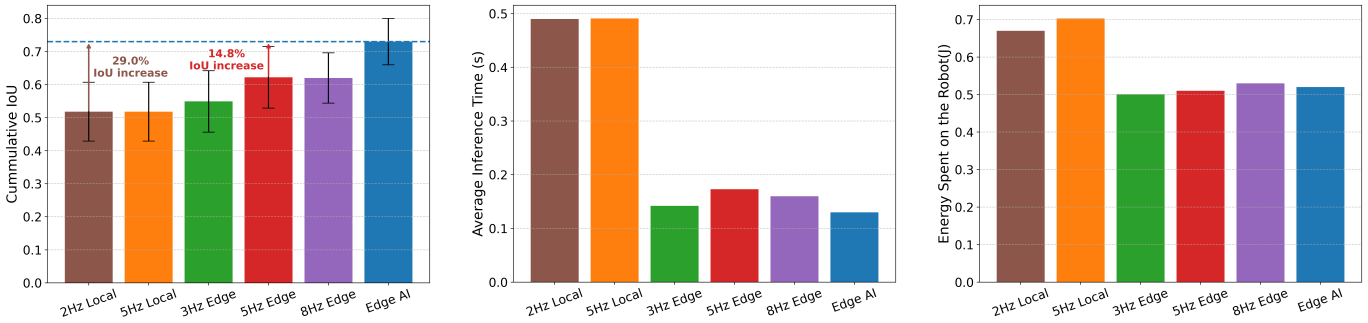
handed over to the edge layer to perform a computationally heavier variant of the object detection algorithm. The AI-assisted task offloading mechanism optimizes the offloading process by considering the state of the robot (battery and network conditions) and deciding the compression and frame rate of the video to be sent to the edge layer.

The **Edge Layer** hosts AI-based applications and deploys a more precise version of the inference model compared to the resource-constrained robot. The additional computing resources include GPUs that speed up inference model execution times. The edge servers are equipped with a GPU to enable the deployment of more precise versions (models equipped with more parameters and layers) of the AI detection applications. Therefore, the edge layer can provide a higher confidence score regarding the detection in the frames sent from the device. However, offloading computations to the edge induces energy costs related to the transmission from the devices to the edge, but enables the execution of more demanding applications.

Finally, the **SMO Layer** is responsible for the multi-cluster orchestration of resources and cross-cluster network connectivity, as detailed in [10]. Specifically, the SMO utilizes Karmada as the multi-cluster orchestration tool and Cilium as a cluster mesh to provide connectivity between the services deployed in different clusters. Each layer under the SMO's control has specific computing capabilities. Therefore, the SMO can manage heterogeneous resources and optimally place application components based on their requirements. The main functionality of the SMO is the construction of a deployment plan for application graphs, propagating the nodes of the graph to the appropriate layers based on the computing capabilities of the clusters and the application components' requirements. The SMO also hosts a monitoring component that continuously observes the system's state and the events generated by the VO. Specifically, the SMO utilizes a central Prometheus instance to aggregate metrics from all other Prometheus instances deployed in the clusters. Upon receiving an event that a person was detected, the SMO can trigger the deployment of the heavier GPU-enabled inference model. As a result, to preserve resources and minimize energy costs, the application nodes at the edge are only deployed if required by the offloading mechanism of the robot in an event-triggered manner.

IV. AI-driven Real-time Inference Optimization

In this section, we analytically describe the AI-assisted task offloading mechanism. As previously mentioned, we consider a battery-powered robot that captures frames using a camera mounted on the robot. As the robot performs person detection and real-time tracking, the remaining percentage of battery power drops. The frames captured by the camera (measured in bytes) can be compressed by the VO before offloading to the edge, reducing the frame size by a compression rate factor to minimize the transmission time. Furthermore, the transmission time to offload the data depends on the current network conditions and the data transmission rate provided by the associated radio access unit. Naturally, the compression rate affects both the transmission time to offload the compressed data and the accuracy of the detection and tracking algorithms, as it lowers the quality of the frames. This behavior is usually captured by a sigmoid function, such as the Gompertz function [11], to indicate that the higher the compression ratio, the lower the expected accuracy due to loss of information per frame. Moreover, the energy consumption of the robot depends on the transmission of the compressed frames and the respective uplink transmission power of the robot. As the overall procedure of the inference pipeline is affected by both the frame rate and the compression rate, we define the confidence score metric. Specifically, the confidence score has a twofold aim: (a) to capture the probability of correctly identifying a person of interest, and (b) to reflect the proportion of useful information provided



(a) Cumulative Average IoU.

(b) Average Inference Time.

(c) Average Energy Consumption on the robot.

Fig. 4: Experiment A: Person detection/tracking evaluation metrics for local execution and different offloaded video frame frequencies for edge execution compared to Edge AI: (a) Cumulative Average IoU, measuring tracking accuracy. (b) Average Inference Time, measuring latency. (c) Average Energy Consumption of the robot.

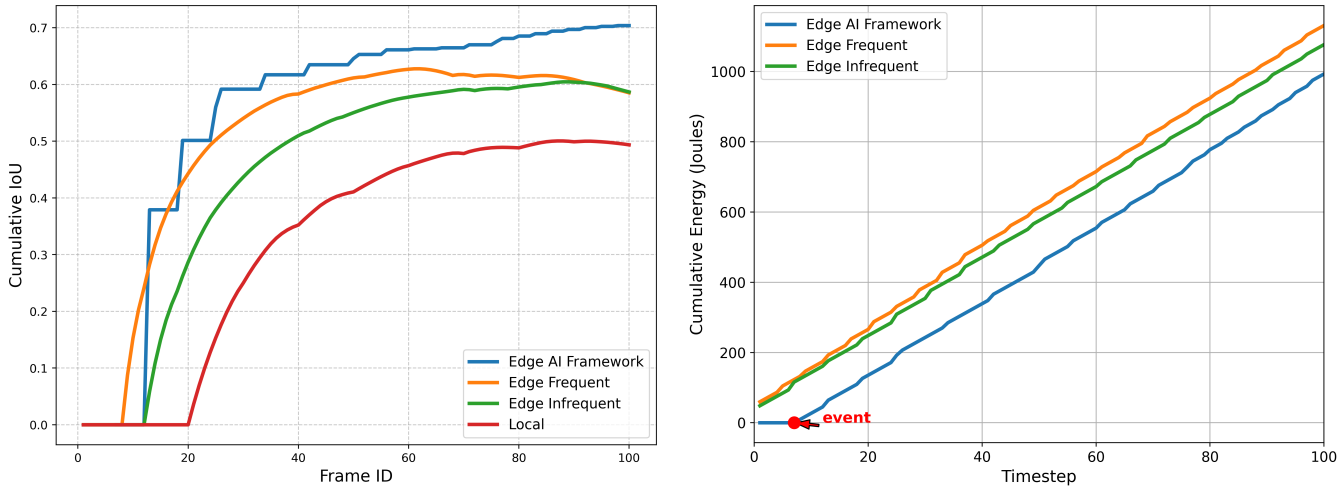
at different frame rates. Higher frame rates allow the robot to capture its surroundings with greater detail, thereby increasing the information content of the offloaded data. In practice, higher confidence scores correspond to more reliable inference results, reducing false positives and false negatives. Based on this modeling approach, we developed an AI-assisted task-offloading mechanism to assist with the robot's real-time decision-making.

We chose to formulate the problem as a Markov Decision Process (MDP) where an RL agent learns to take actions in the environment of the real-time tracking process. Therefore, the VO monitors the information from the network and adapts the offloading decisions in case of an event, each time the robot's state space changes. The state of the RL agent comprises: (a) the data rate with the radio access unit and (b) the remaining battery of the robot. These variables capture the main factors that impact the offloading decision. The data rate determines the inference accuracy, while the remaining battery influences the robot's long term operation. By jointly observing these two variables, the agent can make informed decisions that balance inference accuracy and energy consumption. Considering these two parameters, the robot continuously adjusts the compression rate and the frame rate per second to be offloaded for inference on the edge server, according to the dynamic network conditions and the robot's battery level. The performance of the proposed agent is evaluated by two parameters: (i) the confidence score of the inference process, and (ii) the energy consumption of the robot. Therefore, we define a reward function that needs to be maximized for the robot that tries to strike a balance between these two metrics. The reward design was carefully modeled in order to highlight the trade-off in edge-assisted robotic inference, i.e., maximizing inference accuracy to ensure better image detection outcomes while preserving the robot's battery to prolong the robot's operation. The weights of the reward function prioritize the maximization of the confidence score in order to achieve increased accuracy in the inference process. Moreover, the reward function considers the battery level of each device, ensuring that when the battery is low, the agent prioritizes reducing power consumption. Otherwise, when the battery is high, it focuses on enhancing the performance of the inference process by increasing the frequency of offloaded frames. We used the Proximal Policy Optimization (PPO) algorithm for the RL agent and conducted the training using offline data. The dataset we utilized includes multiple videos from YouTube [12] sent under different bandwidth conditions, emulating the dynamic conditions of a moving robot. The training was performed over 50,000 time steps, at which point convergence was achieved for the reward. To achieve convergence, several runs were executed to fine-tune the hyperparameters of the PPO algorithm. The Edge AI framework incorporated in the AI-assisted Task Offloading mechanism is thoroughly described in Fig. 3.

V. Evaluation Results

To evaluate the proposed Edge AI framework for IoRT, we designed an experimental setup that involved a mobile robotic platform intended for aerial deployment scenarios. The application graph consists of a person detection and tracking pipeline able to run both locally and on the edge server. The event that triggers remote execution and, therefore, on-demand deployment of edge resources is based on *the detection of a person*. As a result, the drone equipped with a camera module monitors the surroundings and offloads the camera frames to the edge when a person is detected. All nodes deployed in the robot utilize Minikube as a Kubernetes-compatible flavor. The SMO and edge server resources were deployed in a small-scale computing infrastructure. All experiments were repeated 10 times, and the results are averaged. Our experimental setup includes the following components:

- *Robot Platform*: A Holybro X500 quadrotor equipped with an NVIDIA Jetson Orin Nano (8 GB RAM). Although the robot is designed for flight, the focus in this analysis excludes this aspect. The drone is equipped with a Luxonis Oak-D pro camera that provides synchronized RGB and depth data streams. Nonetheless, for a fair comparison among the baseline solutions, the MOT20 video dataset has been used [13], with a default FPS of 30Hz. A 5000 mAh 4S1P LiPo battery powers the system,



(a) Cumulative Average IoU.

(b) Cumulative Energy Consumption on the Edge Layer.

Fig. 5: Experiment B: Video segment evaluation metrics for local execution and different offloaded frame frequencies for edge execution compared to Edge AI: (a) Cumulative Average IoU, measuring tracking accuracy. (b) Cumulative Energy Consumption.

and battery discharge along with power consumption is monitored using Nvidia's built-in `tegrastats` utility.

- *Edge Server*: An NVIDIA Tesla T4-equipped Edge Server is part of a small-scale infrastructure. To model the energy consumption of the T4 while running the inference processes, we follow the work in [14]. According to NVIDIA's documentation, the T4 consumes 9W in the idle state and approximately 60W during inference time. As a result, the energy consumption depends on the inference time needed for the offloaded frames.
- *Network Communication*: The robot connects to the Internet via a 5G router to ensure stable and high-throughput communication. The data transmission towards the edge is based on the Zenoh protocol and the Robot Operating System(ROS) - Data Distribution Service bridge from Zenoh, enabling low-latency data streaming of ROS2 messages.
- *Perception Models*: For the inference application node, an object detection model is used, i.e, YOLOv8, whereas the Simple Online and Realtime Tracking (SORT) algorithm [15] is adopted for *person tracking*. For the YOLOv8, we deploy the large model on the Edge and the medium on the robot to achieve a balanced evaluation in terms of inference time. During offloading, images captured onboard are compressed and streamed via Zenoh to a container deployed on a Kubernetes cluster at the edge. There, the image is decompressed before running the inference and tracking algorithms.

A. Evaluation Metrics

The evaluation of the proposed solution focuses on selected parameters:

- *Intersection over Union (IoU)*, also known as the Jaccard index, is the ratio of the *area of intersection* to the *area of the union* between the predicted and ground truth bounding boxes. This gives a quantitative measurement of how well a predicted bounding box aligns with the ground truth bounding box, which fits well within the scope of evaluating a person tracking solution's accuracy.
- *Energy Efficiency for the Robot*: Energy consumption when running the person tracking locally, includes the energy for running the hardware and the person detection and tracking models. Instead, when offloading to the edge, the energy consumption (for the robot) includes the energy consumption to run the hardware, the energy to compress the data, and to send the data over the network (which varies according to the distance from the access point and other network conditions).
- *Inference and Deployment Time*: Time required to perform detection and tracking either locally or remotely. For the on-demand deployment, the container deployment time on the edge is to be taken into account, in addition to the data transmission time, which depends on the actual compression rate and the transmission rate on the communication channel.

B. Performance Evaluation

The first experiment reports a comparative evaluation of the proposed framework against other baseline solutions for a video from the MOT20 video dataset. We chose to compare the framework against five baseline solutions with different configurations regarding the frequency (measured in Hz) and compression rate, namely: (a) 2Hz Local, (b) 5Hz Local, (c) 3Hz Edge with 0.9% compression rate (d) 5Hz Edge with 70% compression rate and (e) 8Hz Edge with 90% compression rate. Note that on-device inference was only possible until 5Hz as after that the device was not able to handle all the images. In Fig. 4 we report the average energy consumption of the robot, the average inference time for the detection and tracking algorithms, and the Cumulative Average of the IoU for all

cases. The proposed solutions, hereinafter referred to as Edge AI, achieve the best cumulative average of the IoU, having 17% better tracking accuracy throughout the experiment than the second best, and especially compared to the solutions executed on the robot as depicted in Fig. 4a. Moreover, Edge AI provides more stable performance regarding the accuracy compared to other methods as it achieves less deviation in the IoU; 8% less compared to the next best baseline method (i.e., 8Hz Edge) and up to almost 25% less. Naturally, the inference time for the Edge-based solutions provides faster results than those executed locally on the resource-constrained robot, presented at Fig. 4b. However, for higher frequency values, the inference time increases slightly for all cases. Regarding the energy consumption of the robot depicted in Fig. 4c, higher frequencies result in higher energy consumption on the robot, i.e., 5% higher energy spent for 5Hz Local compared to 2Hz Local. Nevertheless, offloading data to the Edge Layer also results in a significant power consumption, almost the same as the Edge AI framework, almost 0.51 Joules, indicating that transferring data through the network is also an energy-consuming process. The results indicate that the energy consumption of the robot in the cases of local inference is heavily affected when increasing the frequency of inferred images. Nonetheless, when offloading, the energy consumption of the robot is almost 25% less, although this metric does not account for the energy consumption of the remote inference. Increasing the frequency of the offloaded images (from 3Hz to 8Hz) results in a slight increase in the energy consumption, i.e., almost 5%, that includes the various aspects of the offloading pipeline, i.e., preparation of images, compression, and data transmission. These, results highlight the key role that the AI-assisted offloading mechanism plays in balancing the inference application's performance and the total energy consumption. Finally, we should also mention that the Edge AI framework offloads less data to the edge when the battery of the robots drops, following the rationale of the reward function.

The second experiment reports the Energy Consumption on the Edge Layer and the Cumulative Average of IoU for a realistic experiment of 100 seconds. We chose to compare the Edge AI solution against three other static solutions, namely: (a) Edge Frequent, i.e., 8Hz frequency with 90% compression rate, (b) Edge Infrequent, i.e., 5Hz frequency with 70% compression rate, and (c) Local, i.e., 2Hz frequency. We decided to compare against these policies because they represent indicative scenarios. Higher offloading frequencies improve detection likelihood, while higher compression rates might impact the image quality and lower detection accuracy. In Fig. 5 we present the performance for the on-demand deployment. For the Edge AI solution, the inference is conducted locally on the robot with the same static frequency as the Local baseline. After a person is detected at timestep 7, the VO triggers an event for the on-demand deployment of the inference service on the Edge Layer. On average, the deployment of this service takes 2 seconds, while in the meantime, the robot continues to infer locally. After the successful deployment of the inference application, the AI-assisted Task Offloading algorithm begins to operate. In Fig. 5a, the Cumulative Average of the IoU is presented. The Edge AI framework outperforms all other baseline methods in terms of tracking accuracy, achieving a Cumulative Score for the IoU of 0.7, performing at least 20% than the Edge-based solutions. However, it is clear that before offloading to the Edge, the IoU achieved is significantly lower than the Edge-based solutions, probably due to the less accurate inference model. Moreover, in Fig. 5b, we measure the Cumulative Energy consumption throughout the experiment based on the Tesla T4 utilization and the inference time of each offloaded task. Specifically, the Edge AI framework consumes almost 12% and 8% reduction in terms of Energy Consumed compared to the Edge Frequent and Edge Infrequent cases, respectively. This is mainly due to the on-demand deployment of resources and the adaptive AI-assisted Task Offloading scheme that outperforms the static solutions.

In all cases, the LLM/VLM service is activated for requesting additional information about the visualized frame an additional service is deployed, with an average deployment time of 2s. The user may ask questions like *'Describe the environment'* or *'Is the detected person wearing a hat'* or other questions. For this additional information, an average response time of 0.6s has been registered on the edge, whereas on-device, the LLM/VLM has an average response time of 2s with evident response time savings in performing these requests on the Edge.

VI. Future Perspective

The promising results presented in the previous section make further analysis in wider coverage scenarios of high interest. Some perspectives on enhancements are listed below:

- **Multi-Robot Setting:** Involve multiple robots with different available resources to enable offloading not only at the edge, but also in a peer-to-peer fashion among robots.
- **Security:** Addressing data privacy concerns during data exchange among heterogeneous robots is crucial in IoRT applications, where sensitive information may be transmitted across networks. Therefore, privacy-preserving mechanisms such as end-to-end encryption, robust access control, and FL can play an important role in ensuring secure and trustworthy collaboration between multiple robots.
- **Optimizing/Adapting Edge AI:** Improve AI operations by opting for novel techniques such as speculative decoding to optimize the inference process and adapting the robot's model under changing environments, making use of FL.
- **Complex Environments:** Extend the RL design to handle more complex state spaces (e.g., multi-robot coordination, dynamic obstacles, varying communication quality) and incorporate more sophisticated evaluation metrics such as task success rate, energy efficiency, and safety constraints.

VII. Conclusions

This paper presents an architecture to enable Edge AI in the context of IoRT applications. We introduce an event-

based deployment mechanism tailored for IoRT applications that is provided by an orchestration platform that unifies the management of resources between the robot and the cloud continuum. Moreover, an RL-based agent is formulated to optimize the offloading process of the robot, trying to find a balance between the accuracy of the inference process and the minimization of energy on the robot. Evaluation results indicate a 17% increase for the person and tracking accuracy in energy consumption on the robot, and at least an 8% decrease in utilization of the resources of the edge infrastructure. Regarding our future plans, we aim for a more thorough experimental evaluation of the LLM depending on the capabilities of various resource-constrained devices, and the evaluation in a multi-robot setting controlled by the SMO, allowing for offloading inference tasks in a peer-to-peer network formed between the robots.

ACKNOWLEDGMENT

This research work has received funding from the European Union's Horizon Europe research and innovation programme under the project NEPHELE with grant agreement No 101070487.

REFERENCES

[1] H. Kabir, M.-L. Tham, and Y. C. Chang, "Internet of robotic things for mobile robots: concepts, technologies, challenges, applications, and future directions," *Digital Communications and Networks*, vol. 9, no. 6, pp. 1265–1290, 2023.

[2] O. Vermesan, A. Bröring, E. Tragos, M. Serrano, D. Bacciu, S. Chessa, C. Gallicchio, A. Micheli, M. Dragone, A. Saffiotti, *et al.*, "Internet of robotic things–converging sensing/actuating, hyperconnectivity, artificial intelligence and iot platforms," in *Cognitive hyperconnected digital transformation*, pp. 97–155, River Publishers, 2022.

[3] B. Kar, W. Yahya, Y.-D. Lin, and A. Ali, "Offloading using traditional optimization and machine learning in federated cloud–edge–fog systems: A survey," *IEEE Communications Surveys & Tutorials*, vol. 25, no. 2, pp. 1199–1226, 2023.

[4] A. Imteaj, U. Thakker, S. Wang, J. Li, and M. H. Amini, "A survey on federated learning for resource-constrained iot devices," *IEEE Internet of Things Journal*, vol. 9, no. 1, pp. 1–24, 2021.

[5] L. Militano, A. Arteaga, G. Toffetti, and N. Mitton, "The cloud-to-edge-to-iot continuum as an enabler for search and rescue operations," *Future Internet*, vol. 15, no. 2, p. 55, 2023.

[6] D. Spatharakis, M. Avgeris, N. Athanasopoulos, D. Dechouniotis, and S. Papavassiliou, "Resource-aware estimation and control for edge robotics: A set-based approach," *IEEE Internet of Things Journal*, vol. 10, no. 3, pp. 2003–2020, 2023.

[7] A. A. Arce, N. Filinis, L. Fu, C. Habib, L. Militano, D. Spatharakis, A. Zafeiropoulos, T. M. Bohnert, N. Mitton, and S. Papavassiliou, "Virtual objects for robots and sensor nodes in distributed applications over the cloud continuum," in *2024 IEEE Symposium on Computers and Communications (ISCC)*, pp. 1–6, IEEE, 2024.

[8] D. Spatharakis, I. Dimolitsas, G. Genovese, I. Tzanettis, N. Filinis, E. Fotopoulou, C. Vassilakis, A. Zafeiropoulos, A. Iera, A. Molinaro, and S. Papavassiliou, "A lightweight software stack for iot interoperability within the computing continuum," in *2023 19th International Conference on Distributed Computing in Smart Systems and the Internet of Things (DCOSS-IoT)*, pp. 715–722, 2023.

[9] D. K. Alqahtani, M. A. Cheema, and A. N. Toosi, "Benchmarking deep learning models for object detection on edge computing devices," in *International Conference on Service-Oriented Computing*, pp. 142–150, Springer, 2024.

[10] N. Filinis, I. Dimolitsas, D. Spatharakis, E. Fotopoulou, I. Tzanettis, C. Vassilakis, A. Zafeiropoulos, and S. Papavassiliou, "A synergetic meta-orchestration framework for distributed application deployments in the computing continuum," in *Proceedings of the 1st International Workshop on MetaOS for the Cloud-Edge-IoT Continuum*, MECC '24, (New York, NY, USA), p. 21–27, Association for Computing Machinery, 2024.

[11] X. Zhao, M. Hosseinzadeh, N. Hudson, H. Khamfroush, and D. E. Lucani, "Improving the accuracy-latency trade-off of edge-cloud computation offloading for deep learning services," in *2020 IEEE Globecom Workshops (GC Wkshps*, pp. 1–6, 2020.

[12] F. Loh, F. Wamser, F. Poignée, S. Geißler, and T. Hoßfeld, "Youtube dataset on mobile streaming for internet traffic modeling and streaming analysis," *Scientific Data*, vol. 9, p. 293, Jun 2022.

[13] P. Dendorfer, H. Rezatofighi, A. Milan, J. Shi, D. Cremers, I. Reid, S. Roth, K. Schindler, and L. Leal-Taixé, "Mot20: A benchmark for multi object tracking in crowded scenes," *arXiv preprint arXiv:2003.09003*, 2020.

[14] Q. Zeng, Y. Du, K. Huang, and K. K. Leung, "Energy-efficient resource management for federated edge learning with cpu-gpu heterogeneous computing," *IEEE Transactions on Wireless Communications*, vol. 20, no. 12, pp. 7947–7962, 2021.

[15] N. Wojke, A. Bewley, and D. Paulus, "Simple online and realtime tracking with a deep association metric," in *2017 IEEE international conference on image processing (ICIP)*, pp. 3645–3649, IEEE, 2017.

This is the author accepted manuscript version of an article published in the IEEE Internet of Things Magazine. The Version of Record is available at https://doi.org/10.1109/MIOT.2026.3655981.

© 2026 IEEE. Personal use of this material is permitted. Permission from IEEE must be obtained for all other uses, in any current or future media, including reprinting/republishing this material for advertising or promotional purposes, creating new collective works, for resale or redistribution to servers or lists, or reuse of any copyrighted component of this work in other works.